

Estructura del Firmware

Variables declaradas en app_common.h :

Estas variables se declaran aqui para que cualquier otro archivo del proyecto puedan utilizarlas. las variables creadas aqui son:

Definiciones de prioridades de tareas y stack:

CONSTANTES	VALOR	DESCRIPCIÓN
TASK_USER_CONTROLS_STACK	2048	tamaño del stack en bytes
TASK_MENU_CONFIG_STACK	2048	
TASK_LCD_DISPLAY_STACK	2048	
TASK_SWEEP_STACK	2048	
TASK_UART_STACK	2048	
TASK_NVS_STACK	2048	
TASK_USER_CONTROLS_PRIORITY	2	Prioridad de tareas
TASK_MENU_CONFIG_PRIORITY	3	
TASK_LCD_DISPLAY_PRIORITY	2	
TASK_SWEEP_PRIORITY	2	
TASK_UART_PRIORITY	2	
TASK_NVS_PRIORITY	1	
QUEUE_MENU_EVENTS_LEN	8	tamaño de nodo de colas
QUEUE_SWEEP_CMD_LEN	4	
QUEUE_DISPLAY_LEN	8	
QUEUE_NVS_CMD_LEN	4	
QUEUE_UART_RX_LEN	16	

Colas	Descripción	tareas que lo utilizan
queue_menu_events	cola que transporta los eventos generados por el menú/controles de usuario (toques de pantalla o pulsaciones de botón ya traducidos a "eventos de menú").	task_user_control task_menu_config task_uart

queue_sweep_cmd	cola que recibe comandos hacia la tarea de barrido (sweep), seguramente para iniciar/detener o configurar parámetros del DDS (frecuencia, amplitud, etc.).	task_sweep task_menu_config
queue_display	cola que lleva los datos o mensajes que la tarea de LCD debe mostrar en pantalla.	task_lcd_display task_menu_config
queue_nvs_cmd	cola con comandos/valores a guardar en memoria no volátil (NVS).	task_menu_config task_nvs
queue_uart_rx	cola que almacena los bytes recibidos por UART.	task_uart

Semáforos:

SEMÁFORO	DESCRIPCIÓN
sem_btn_press	semáforo binario, activado por una interrupción de GPIO cuando se presiona un botón físico, para despertar la tarea correspondiente sin hacer polling.

Mutex

Mutex	DESCRIPCIÓN
mutex_uart_tx	mutex para evitar que dos tareas escriban por UART al mismo tiempo y se mezclen los datos.

Definición de pines para LCD

CONSTANTE	PARÁMETRO	DESCRIPCIÓN
LCD_SCLK	GPIO_NUM_39	Serial Clock: el reloj del bus SPI. El ESP32 lo pulsa para marcar el ritmo de la transferencia; en cada pulso el ST7789 lee o envía un bit.
LCD_MOSI	GPIO_NUM_38	Master Out Slave In: la línea de datos. Por acá viajan los píxeles, comandos y configuraciones desde el ESP32 hacia la pantalla. No hay MISO porque el ST7789 es un dispositivo "write-only" en este proyecto (no se le consulta nada, solo se le envía).
LCD_CS	GPIO_NUM_45	Chip Select: habilita o deshabilita la pantalla en el bus SPI. Se pone en bajo (0) justo antes de cada transferencia y vuelve a alto (1) al terminar. Sirve para que varios dispositivos puedan compartir el mismo

		bus SPI sin interferirse (en este proyecto el AD9833 también usa SPI, pero por SPI2, mientras la pantalla usa SPI3).
LCD_DC	GPIO_NUM_42	Data/Command: le indica al ST7789 si lo que está llegando por MOSI es un comando (configuración, como "inicializar", "setear área de dibujo") o un dato (valores de píxeles). En bajo = comando, en alto = dato.
LCD_RST	GPIO_NUM_0	Reset: reinicia el controlador del panel a su estado por defecto. Se pulsa brevemente al arrancar (en <code>esp_lcd_panel_reset()</code>), antes de inicializarlo con la configuración del proyecto.
LCD_BL	GPIO_NUM_1	Backlight: controla la retroiluminación de la pantalla. En el código se configura como salida GPIO simple y se pone en alto (<code>gpio_set_level(LCD_BL, 1)</code>) para encender la luz.
LCD_H_RES	320	resolución horizontal, 320 píxeles de ancho.
LCD_V_RES	240	resolución vertical, 240 píxeles de alto.
TOUCH_SDA	GPIO_NUM_48	Serial Data: línea bidireccional por donde viajan los datos entre el ESP32 y el CST816S. A diferencia del SPI, por acá puede responder el chip (por eso sí se necesita comunicación en ambas direcciones).
TOUCH_SCL	GPIO_NUM_47	Serial Clock: el reloj del bus I2C, generado siempre por el maestro (el ESP32). Sincroniza cuándo se lee cada bit en SDA.
TOUCH_INT	GPIO_NUM_46	Interrupt: cuando el usuario toca la pantalla, el CST816S pone este pin en bajo para avisarle al ESP32 que hay un evento táctil disponible para leer. Esto evita que el ESP32 tenga que estar preguntando constantemente "¿hubo toque?" (polling) — en cambio espera la interrupción y sólo actúa cuando hay algo.

Buses utilizados para la comunicación entre el ESP32 y el Display Táctil.

BUSES	DESCRIPCIÓN
SPI3	Se utiliza para dibujar píxeles en el panel (ST7789), que es rápido y unidireccional.
I2C	Utilizado para leer eventos táctiles del CST816S, que es más lento pero suficiente porque solo manda coordenadas de toque.

Handle para controlar el display Táctil

Handle	DESCRIPCIÓN
io_handle	Handle de la interfaz de comunicación con el panel (en este caso SPI), necesario para enviar comandos/datos a la pantalla.
panel_handle	Handle del controlador del panel físico (driver ST7789), representa la pantalla en sí.
disp_handle	Puntero al "display" dentro de LVGL (la librería gráfica), que conecta el panel físico con lo que se dibuja en la interfaz.
i2c_bus	Handle del bus I2C maestro, usado para comunicarse con el controlador táctil.
tp_handle	Handle del driver del touch capacitivo (CST816S), representa el sensor táctil.

Control del DDS ad9833

Handle	Descripción
spi_handle	Handle del dispositivo SPI que representa al AD9833 en el bus SPI2 (bus distinto al de la pantalla, que usa SPI3). Se usa para enviarle los 16 bits de cada transacción.

control_word : Para este caso, la palabra de 16 bits que representa el estado actual del registro de control del AD9833 .

Se inicializa con los bits B28 y RESET activados, y después se va modificando bit a bit en ad9833_enable_output() / ad9833_disable_output() para prender o apagar la salida sin tocar el resto de la configuración.

Bits de direccionamiento

BIT	DESCRIPCIÓN
AD9833_REG_CTRL	Ocupan los 2 bits más altos de cada palabra de 16 bits enviada, e indican a qué registro interno del chip va dirigido ese dato (control, frecuencia o fase).
AD9833_REG_FREQ0	

Bits de control

BIT	DESCRIPCIÓN
AD9833_CTRL_B28	Se combinan para formar control_word
AD9833_CTRL_RESET	
AD9833_REG_CTRL	

Palabra de Control:

AD9833_MCLK_HZ: la frecuencia del cristal del propio módulo AD9833 (25 MHz), que se usa en `ad9833_set_freq()` para calcular el "frequency word" de 28 bits que determina la frecuencia de salida real.

GPIO AD9833

PARÁMETRO	GPIO	DESCRIPCION
AD9833_PIN_CLK	GPIO_NUM_12	SCLK SPI2 (Serial Clock): el reloj del bus, equivalente al SCLK de la pantalla. El AD9833 muestrea cada bit en el flanco de bajada , esto lo define AD9833_SPI_MODE 2 (CPOL=1, CPHA=0): el reloj arranca en alto y los datos se leen cuando baja.
AD9833_PIN_DATA	GPIO_NUM_11	SDATA SPI2 (Serial Data): la línea de datos, equivalente al MOSI. Por acá llegan las palabras de 16 bits con los registros de control y frecuencia. No hay MISO por la misma razón que en la pantalla.
AD9833_PIN_FNC	GPIO_NUM_10	Chip Select SPI2Function/Chip Select: equivalente al CS de la pantalla. Se pone en bajo antes de cada transferencia de 16 bits y vuelve a alto al terminar. El AD9833 ignora cualquier dato en DATA mientras FNC esté en alto.

PARÁMETRO	DESCRIPCIÓN
AD9833_SPI_CLK_HZ	Frecuencia del bus SPI2
AD9833_SPI_MODE	
AD9833_SPI_CLK_HZ	Velocidad del bus 1Mhz

Bits de dirección

PARÁMETRO	DESCRIPCIÓN
AD9833_REG_CTRL	ocupan los 2 bits más altos de cada palabra de 16 bits enviada, e indican a qué registro interno del chip va dirigido ese dato (control, frecuencia o fase).
AD9833_REG_FREQ0	